

Access Modifiers & Constructors (Questions I had in my mind)

Q What convention should be followed on applying access modifiers on classes & it's members while developing an application?

Ans. Provide the class & it's members the most restrictive access which still allows the design to work.

In a nut-shell

- mostly private, only widen access when there is clear reason
- public → only for types that form part of the application's API
 - ↳ they are long term commitments, changing / restricting later might break callers
- only expose behaviour not internal state
- prefer immutability where practical

Q When an application is run, exactly when is a class loaded & it's static members initialized?

Ans A very high-level abstract explanation for now -

JVM treats class handling in three separate stages

1. Loading → finding the .class file & creating JVM's internal state
2. Linking → verifying it's bytecode & initializing it's static fields with default values
3. Initialization → executing static field initializers & static {} blocks

→ import statement does not load the class

When is class initialised?

- 1) Creating first object
- 2) Calling a static method
- 3) Reading a non-constant static field
- 4) Assigning a static field

For now, this much information is enough



Additional Bullets

→ instance variable with final keyword must be initialized immediately during declaration or within a constructor

→ `object.toString()` doesn't need to be explicitly invoked.
`System.out.println(object)` will implicitly invoke the `toString` method of that object or its parent.